

xMCF

ISO/PAS 8329

Description of mechanical connections
and joints in structural systems

ISO/PAS 8329 xMCF WG meeting
focus group on issues [#105](#) / [#61](#)
2026-03-11

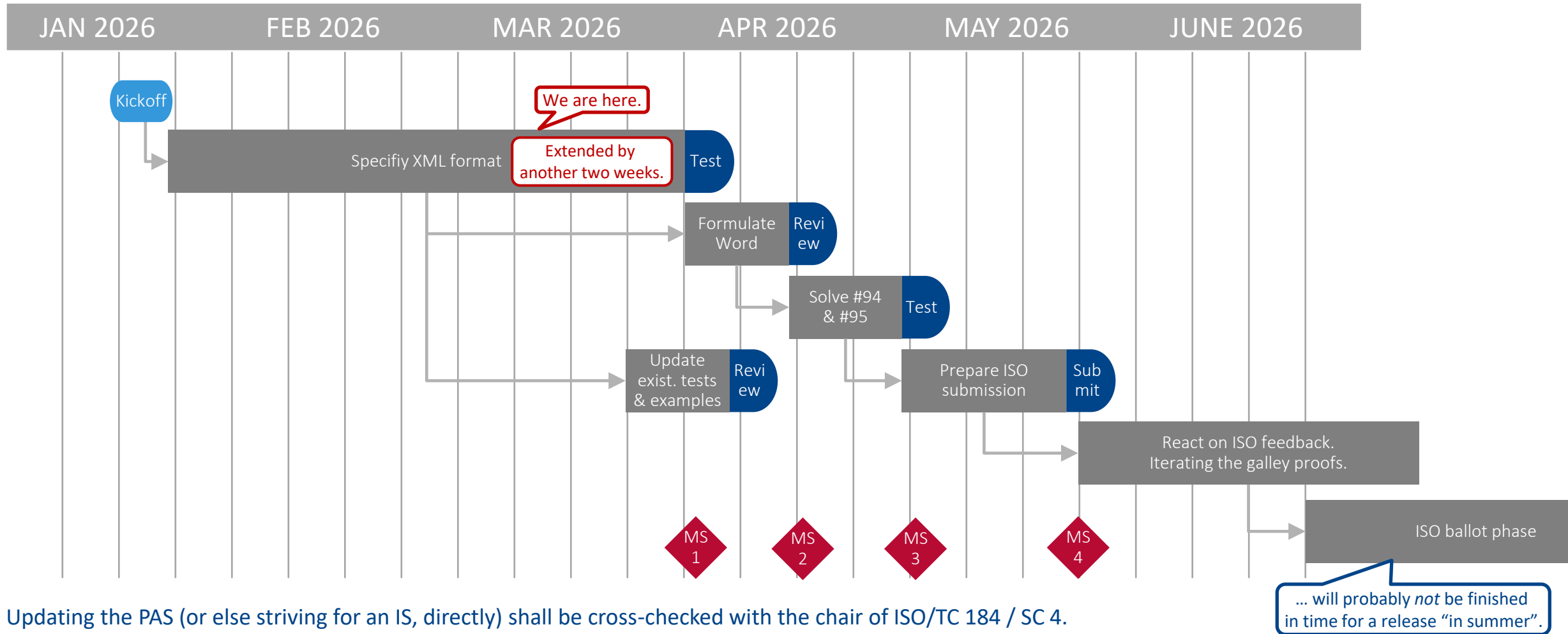
Agenda

As announced at https://github.com/economidis-nick/createXSDforxMCF/blob/master/VDA_FAT_AK_25/Meetings/2026-03-11_VideoConference/README.md

- Welcome & Flying over today's topics
- Timeline for next release
- Results of last PPC (if any)
- Short recap on [Schematron](#) and its priority
- How do we design templates in the XSD file? — Nick's [reply](#) to Stephan's [comment to issue #105](#)
- Focus on detailing the XML elements, based at issue [#105](#)
- Steps we should take next



Timeline for a release in the summer



Updating the PAS (or else striving for an IS, directly) shall be cross-checked with the chair of ISO/TC 184 / SC 4.
Next meeting of the Policy and Planning Committee (PPC) was scheduled for 18th of Feb.

Leveraging Schematron XML verification

Homework: Please reflect on the following questions until the next meeting:

- How can we leverage the “[power of Schematron](#)” for (which of) our purposes?
- What priority would we assign to setting up this means of verification?

Decision, 2026-03-11:
We will care for the XSD file, first, and then see, whether we need Schematron capabilities, on top.

We consider providing one or several Schematron files to the next or to a following publication of χ MCF.

What are relevant scenarios / use cases?

- Developing export algorithms & verification of the results.
- Quality assurance when receiving data from (external) partners.
- Further development of the standard. Verification of examples. Development of recommended practices.
- Definition of different, increasingly strict rules along the product development process:
At the beginning of the PDP, χ MCF may have many “gaps,”
but towards the end, many optional attributes should be provided.

How do we design templates in the XSD file?

Stephan's [comment to issue #105](#)

Quote:

„It is not possible to define two different `<xs:element name="connection_0d"/>` inside the same scope. This means, we have to use the same `xs:element` for both, the `<connection_template/>` and a standard definition inside a `<connection_list/>`. This subsequently means that we have to remove anything from the `<connection_0d/>` that is mandatory, e.g. the `"connection_0d_type"`.“

How to handle this?

→ BETA CAE announced that it will propose a solution. (Homework done).

Addendum, 2026-02-28: See according [comment in issue 105 of the very same evening](#). (To be discussed.)

Issue is that mandatory attributes become optional, if a connection refers to a template, which provides the attribute.
How to represent this in XSD?

Sub-type declaration to be separated from “simple” attributes?

Assumption: XSD will not be able to help, once we introduce *external* templates.

To be verified: Can Schematron (or XSD) open an *external* template via e.g. an URL?

→ Homework / Thomas to involve Kevin.

Detailing XML Elements — Template Syntax, Screw Example

We identified the example XML file [chapter 9 5 7 exampleA patched for xMCF3.2.xml](#) (provided by the comment of [2026-10-14 09:17+0100](#)) as a good starting point.

If yes, the template must *not* have an attribute “**label**”, since this would usually be *unique to each single* connector instantiated from the template.

It suggests as a syntax for defining an (file-embedded) list of *connection templates* at XML root level:

```
<connection_templates>
```

```
<connection_0d label="screw template 4711">
```

For `<washer/>` and we introduced the new attribute “`template_label`”. Shouldn’t we use it here, too? (I

Decision, 2026-03-11:
We change to „`template_label`” for all kind of templates.
→ Draft example files to be updated.

```
<threaded_connection part_code="M10x50 12.9" length="50." diameter="10"
```

```
head_diameter="16.0" head_height="5" sink_size="4" thread_length="35" >
```

```
<!-- Attributes "torque", "angle" and "pretension" must not appear in a template! -->
```

```
<screw /> <!-- Screw may come without any attributes -->
```

```
<washer outer_diameter="20"/>
```

```
</threaded_connection>
```

```
</connection_0d>
```

```
</connection_templates>
```

Homework: Consider what “misuse” or contradictory set of data can occur by this approach!

Detailing XML Elements — Reference Syntax, Screw Example

The example suggests as syntax for *referring to* a connection template at `<connection_0d/>` level:

Default form — no change to / overwriting template data:

```
<!-- Screw exactly as defined in connection_templates: -->
<connection_0d label="SCREW_100532" template="screw_template_4711">
  <threaded_connection>
    <normal_direction x="2.9" y="0.1" z="0.0" />
  </threaded_connection>
  <loc> 1500.3809 838.75885 730.6529 </loc>
</connection_0d>
```

Minimal change — overwriting an attribute value:

```
<!-- Screw exactly as defined in connection_templates, but with smaller torque: -->
<connection_0d label="SCREW_100533" template="screw_template_4711">
  <threaded_connection torque="70" >
    <normal_direction x="3.0" y="0.0" z="0.0" />
  </threaded_connection>
  <loc> 1540.3809 838.75885 730.6529 </loc>
</connection_0d>
```



Detailing XML Elements — Reference Syntax, Screw Example

The example suggests as syntax for *referring to* a connection template at `<connection_0d/>` level:

Overwriting a child element:

```
<!-- Screw exactly as defined in connection_templates, but with larger washer: -->
<connection_0d label="SCREW_101537" template="screw_template_4711">
  <threaded_connection>
    <normal_direction x="3.1" y="-0.1" z="0.0" />
    <washer outer_diameter="30"/>
  </threaded_connection>
  <loc> 1580.3809 838.75885 730.6529 </loc>
</connection_0d>
```

Attention: Here, only the `outer_diameter` of the `<washer/>` is overwritten. Any other washer-parameter given from the template stays effective.

Detailing XML Elements — Template Syntax, Bolt Example

The example extends to cases of *deeper structured* elements:

```
<connection_templates>
```

```
<connection_0d label="bolt_template_4712">
```

```
<threaded_connection part_code="M10x50 12.9" length="50" diameter="10"
```

```
  head_diameter="16.0" head_height="5" sink_size="4" thread_length="35" >
```

```
  <!-- Attributes "torque", "angle" and "pretension" must not appear in a template! -->
```

```
  <normal_direction x="0" y="0" z="-10"/>
```

```
  <washer outer_diameter="20" inner_diameter="10.3" thickness="1.5" attached="true"/>
```

```
  <!-- Washer attached to the screw head -->
```

```
  <bolt>
```

```
    <nut diameter="16." height="5" />
```

```
    <!-- Friction attributes must not appear in a template,  
         unless it is against an attached washer -->
```

```
    <!-- Attributes "clipped_to" or "fixed_to" must not appear in a template,  
         since they each contain a flange partner index! -->
```

```
  </bolt>
```

```
</threaded_connection>
```

```
</connection_0d>
```

```
</connection_templates>
```

For <washer/> and <nut/>, we introduced the new attribute "template_label". Shouldn't we use it here, too? (Instead of "label"). (Cf. slide 6.)

Detailing XML Elements — Reference Syntax, Bolt Example

The connection template can be *referred to* for example like this:

```
<!-- Bolt exactly as defined in connection_templates, but with less torque, larger washer & nut: -->
```

```
<connection_0d label="BOLT_100732" template="bolt_template_4712">
```

```
  <threaded_connection torque="70">
```

```
    <normal_direction x="3.1" y="-0.1" z="0.0"/>
```

```
    <washer outer_diameter="30"/>
```

Check, whether a washer may refer to a connection template, too! → we need examples to better understand. → next slides!

```
    <bolt>
```

```
      <nut height="8"/>
```

Check, whether a nut may refer to a connection template, too! → we need examples to better understand. → next slides!

```
    </bolt>
```

```
  </threaded_connection>
```

```
</connection_0d>
```

Rule (→ Word file): If the connection template is a `<screw/>`, the referring element must not redefine it to a `<bolt/>` and vice versa!

To be extended to similar situations in case of future extensions of χMCF, e.g. a spotweld (from template) must not be redefined to a gumdrop!

Detailing XML Elements — Template Syntax for a Washer

Draft example:

```
<!-- The new parameter "template_label" is allowed for a template, only. -->
<washer template_label="template for a loose washer"
  outer_diameter="20" inner_diameter="10.3" thickness="1.5"
  strength_property_class="12.9" part_code="M10x20x1.5 12.9" />
<!-- Washer next to nut would have friction to last part, which is unknown. Thus, not allowed. -->
<!-- A washer can be attached to a screw head or a nut.
      But here as a template without context, one can't know. Thus, "attached" is not allowed. -->
```

Remarks:

- Templates must not contain friction parameters, because friction always is a physical property of a *combination* of two items.
 - We should differentiate between “firmly fixed” washers (i.e. washer and nut/screw form one solid) and “rotate-able” ones, where the washer and nut/screw can rotate independently around their common axis.
- Remark: BETA CAE’s study (2025-02-18) showed that we do *not* need a new attribute, here.

Detailing XML Elements — Template Syntax for a Nut w/ Washer

Draft example:

```
<!-- The new parameter "template_label" is allowed for a template, only. -->
<nut template_label="template for a nut with an attached washer"
  diameter="16." height="5" strength_property_class="12.9" part_code="M10x5 nut 12.9"
  static_friction="0.8" kinetic_friction="0.6" >
  <!-- Attributes "torque" and "angle" must not appear in a template! -->
  <!-- Attributes "clipped_to" or "fixed_to" must not appear in a template,
    since they each contain a flange partner index! -->
  <!-- The washer of this nut is attached to it in such a way that it can rotate.
    Therefore, the nut's friction against the washer is meaningful and allowed. -->

  <!-- If the washer is part of or firmly attached to the nut, it cannot have a part code! -->
  <washer outer_diameter="25" thickness="1.5" strength_property_class="12.9"
    attached="true" static_friction="0.7" kinetic_friction="0.5"/>
  <!-- The friction of the washer describes the contact to the connected part. -->
  <!-- This washer could reference a template, itself. -->
</nut>
```

Remark: Friction between nut and attached but rotate-able washer is allowed!

Detailing XML Elements — Template Syntax for a Bolt, using Washer & Nut templates, itself.

Draft example:

For `<washer/>` and `<nut/>`, we introduced the new attribute `"template_label"`. Shouldn't we use it here, too? (Instead of `"label"`). (Cf. slide 6.)

```
<!-- The next bolt reference references washer and nut templates, itself: -->
<connection_0d label="bolt_template_4733">
  <threaded_connection length="50" diameter="10" head_diameter="16" head_height="5" thread_length="35"
    part_code="M10x50 12.9">
    <!-- Attributes "torque", "angle" and "pretension" must not appear in a template! -->
    <normal_direction x="0" y="0.8" z="-1.2"/>
    <washer template="template for a loose washer"/> <!-- The washer next to the bolt's head. -->
    <bolt>
      <nut template="template for a nut with an attached washer"/><!-- Nut has attached washer.-->
    </bolt>
  </threaded_connection>
</connection_0d>
```

Detailing XML Elements — Template Syntax, Gumdrops Example

(new — see file “chapter_10_5_exampleB__with_templates.xml”)

For a gumdrop, the example template would be:

```
<connection_templates>
  <connection_0d label="GumDrop template">
    <!-- Assumed Unit system with mass attribute with value="kg" -->
    <gumdrop diameter="5.0" mass="0.0033" material="Glue_Material" />
  </connection_0d>
</connection_templates>
```

For <washer/> and <nut/>, we introduced the new attribute “template_label”. Shouldn’t we use it here, too? (Instead of “label”). (Cf. slide 6.)

Detailing XML Elements — Reference Syntax, Gumdrops Sequence

(new — see file “chapter_10_5_exampleB__with_templates.xml”)

For a gumdrop sequence, the example reference could be (as suggested in [GitHub comment](#)):

```
<!-- Sequence of gum drops exactly as defined in connection_templates: -->
<connection_1d label="DROP_LINE_11000" template="GumDrop template">
  <sequence_connection_0d spacing="30.0" margin="1.0" priority="spacing">
    <gumdrop/>
  </sequence_connection_0d>
  <loc_list>
    ...
  </loc_list>
</connection_1d>
```

Observation (CF): A template for a 0d item as parameter of a 1d item appears inappropriate to me.

What about this approach:

```
<connection_1d label="DROP_LINE_11000" template="Later, here could be place for a sequence template.">
  <sequence_connection_0d spacing="30.0" margin="1.0" priority="spacing" template="GumDrop template">
```

Decision, 2026-02-25:

<sequence_connection_0d/>
will *not* be considered within the scope
of our current work, just as has been
decided for all the other
<connection_1d/> types, before.

XML Example File provided by [@ghirel](#) / PDM-IF.

The example “SAMPLE-FASTENER_003--.--1In Work000-ACOMMON_ENG_01_xmcf3.2 GHi.xml” has shown us that:

- `<loc/>` must not appear inside `<connection_templates/>`.
- `<stacking/>` must not appear inside `<connection_templates/>`.

Contentual Action Items $\triangleq$ Next Steps:

▪ Standard document (Word file):

- Description of overall concept & objectives, intended application & motivation etc.
- Description of semantics for internal catalog
- Description of semantics for reference to a connection template
- Definition of example use cases and in-document XML-examples

▪ XML & XSD — See discussion in [#105](#)!

- Sketching XML example file(s), incl. (w.i.p.)
 - precedence rule (w.i.p.) and dropped mandatory attribute,
 - internal (w.i.p.) and external file-based catalog
- Description of syntax for internal catalogs (w.i.p.)
- Description of syntax for reference to a connection template:
 - internal (w.i.p.), external file-based, external database-based

- Drafting XML schema (XSD) and test cases / test example files

Homework: Collect use cases which can yield e.g. test cases, shared as Markdown files in GitHub at https://github.com/economidis-nick/createXSDforxMCF/tree/issues_61_94_95_105/VDA_FAT_AK_25/Meetings/2026-02-18_VideoConference

- Investigate xMCF elements and attributes for eligibility for template.
⇒ we need a rule, here! ⇒ Homework!



Next meetings

(2026-03-18 — CF on business journey)

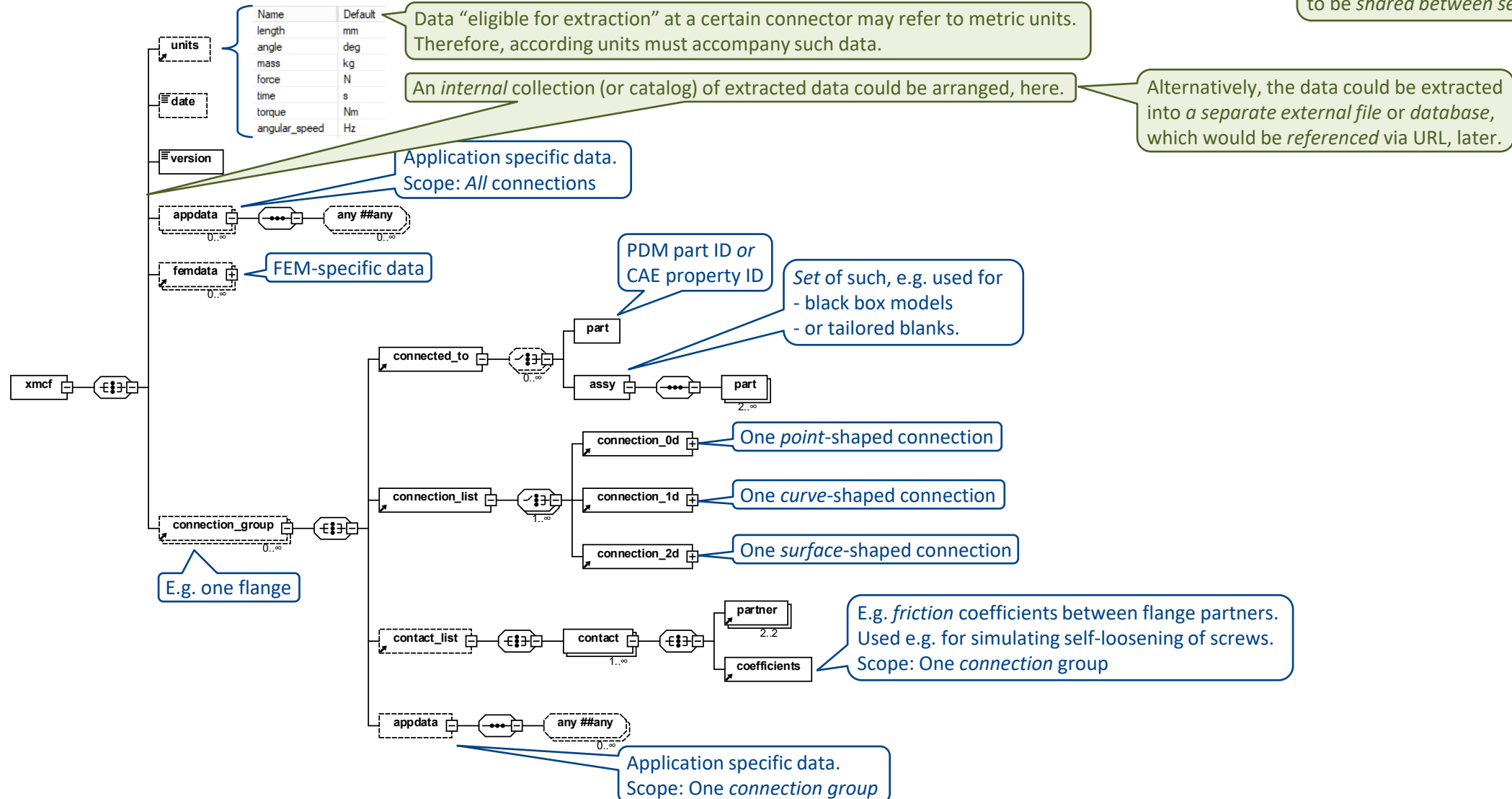
- 2026-03-25, 15:00 CET — focus group on [#105](#) / [#61](#)
— objective: agree on XML format suggestions
- 2026-04-01, 15:00 CEST — full group
— objective: agree on XML format suggestions

Shall we continue with April 8., 15., 22.? — Yes.

CF sent / will send out invitations.



Structure of χ MCF 3.1.1 XSD – High Level



“eligible for extraction” to be understood as:
Data can be *taken out of one* specific connector,
to be *shared between several* connectors.

